Distracted Drivers

By: Andrea, Payal, Andre, Eldar, Katherine, Cameron

“Distracted driving is dangerous, claiming 3,522 lives in 2021” - United States Department of Transportation

The simple act of changing the music channel, checking your phone, or making a call can result in a life changing event for victims and the families of victims of car accidents.

Our project is based around detecting different actions of distracted driving so we are able to alert the driver of the distraction. We are looking to build a model with the highest accuracy in detection to hopefully prevent future accidents.

NEW YORK

3 Killed in Car Accidents in Brooklyn and Queens

Donald Hennessy is killed when struck by car while crossing 56th Street in Queens (NYC) hit-and-run accident; two separate car accidents leave two others dead; photo (M)

By Michael Wilson

PRINT EDITION 3 Killed in Car Accidents In Brooklyn and Queens | March 14, 2005, Page B00003

AFRICA

At Least 24 Killed in Minibus Crash in Morocco

The vehicle overturned at a curve on its way to a weekly town market southeast of Marrakesh, officials said.

By The New York Times

NEW YORK

A Car Accident Throws a Productive Life Into Turmoil

David Nublett prided himself on being self-sufficient. But after being hit by a car, he not only lost his ability to work, but also his independence and his savings.

By John Otis

01
Building a Model

What is Our Data Source?

What Causes Distracted driving?:

1) Drinking Coffee
2) Tuning/changing the radio station
3) Using the mirror

What Doesn't Cause Distracted driving?:

1) Attentive Driving

Looking at Our Data and Finding Differences

1) Import images in a data folder. The labels were Attentive, UsingRadio, DrinkingCoffee, and UsingMirror.

Used metadata function that identifies class and the splits (training or testing; The data are images)

2) The identified splits covered all classifications. (80% train & 20% test)

Using sns.countplot(x= 'split', data= metadata), we created the graph

3) By splitting our data for each class we were able to see they each had 80% training and 20% testing.

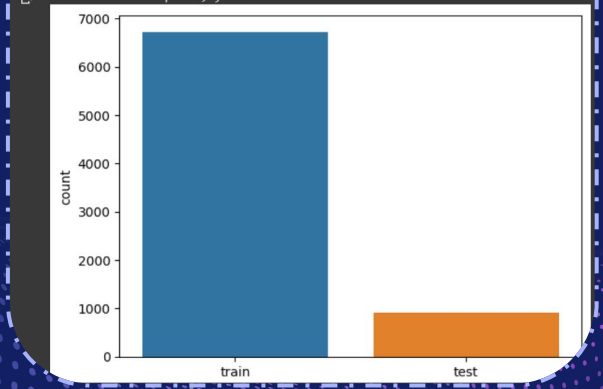
We split our data by using hue= 'split'

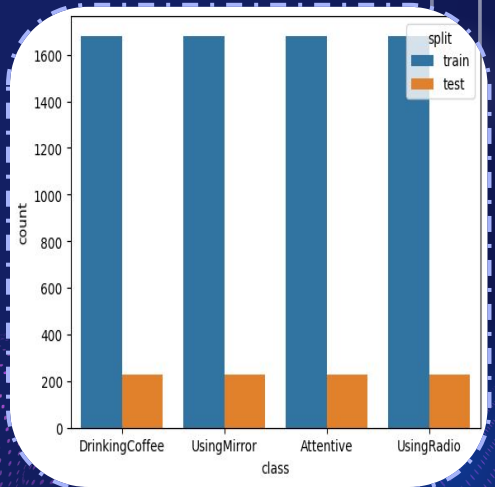
We also want to keep the training and testing balanced to remove bias

```
# get a table with information about ALL of our images
metadata = pkg.get_metadata(metadata_path)

# what does it look like?
metadata.head()

# Uncomment the function below to check the bottom five!
# metadata.tail()
```

	class	split	index
4780	DrinkingCoffee	train	4780
4295	DrinkingCoffee	train	4295
2333	DrinkingCoffee	train	2333
3899	DrinkingCoffee	train	3899
2215	DrinkingCoffee	train	2215

```
### YOUR CODE HERE
sns.countplot(x= 'split', data= metadata)

### END CODE
```



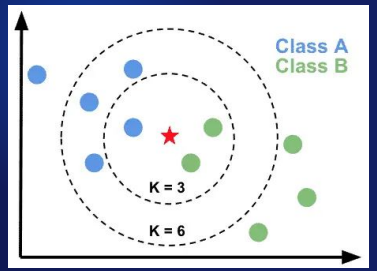


Choosing the Model

- Inputs were images taken by cameras inside of cars that showed attentive and distracted drivers
- Outputs were accuracy scores of how well the system predicted the images into the four types of driver situations
- Below are the models we experimented with

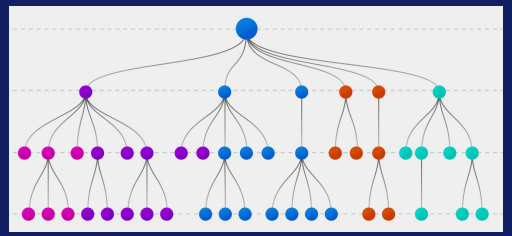
KNeighbors:
Looks at a given number of similar images surrounding the input image and outputs the category it is most surrounded by

50.8%



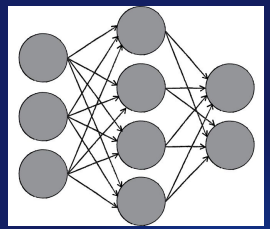
Decision Tree:
Makes one decision or another based on a set of rules it is given

24.6%



Neural Networks:
Processes images pixels with different weighted filters and outputs the category it thinks the image best fits into

55.7%



02
Neural Networks

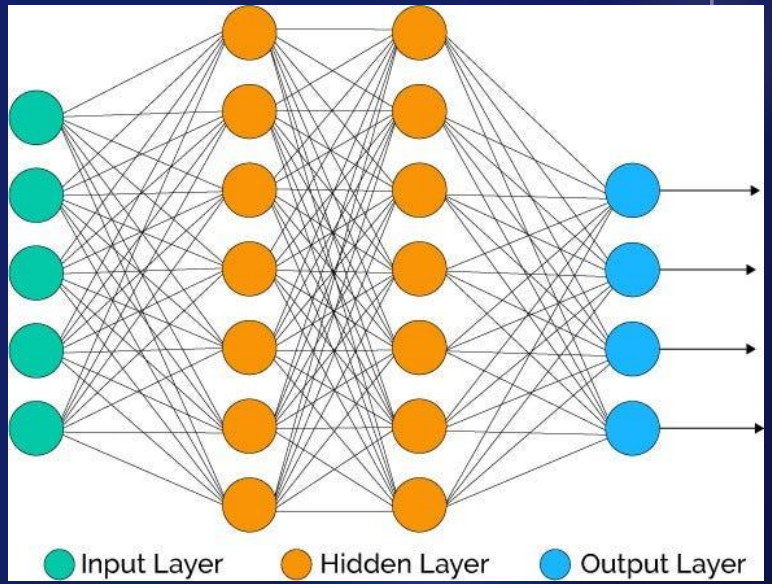
Understanding and building Neural Networks

A neural network is a computing model that uses algorithms to recognize patterns in data.
When building a neural network, we need to pay attention to:

- The number of neurons
- The activation of the neurons
- The losses and metrics

Taking into account the information above, we needed to:

1. Specify the model
2. Add layers to the network
3. Turn the model on by compiling it



Applying Neural Networks

First, we built our model. Next, we augmented the data, trained our model, and then fit our data.

```
model_3 = Sequential([Input(shape = (64, 64, 3)),
                      Flatten(),
                      Dense(units=128, activation = 'relu'),
                      Dense(units=4, activation = 'softmax')])
model_3.compile(loss='categorical_crossentropy',
                optimizer = optimizers.SGD(learning_rate=1e-4, momentum=0.95),
                metrics = ['accuracy'])
```

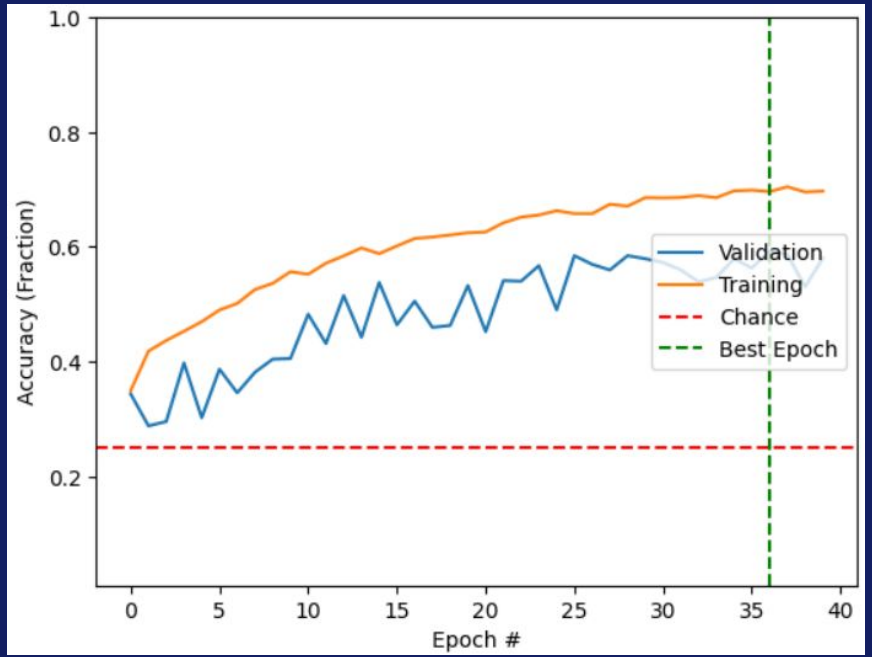
```
data_augmentation = ImageDataGenerator(
    rotation_range=10,  # Rotate images randomly up to 10 degrees
    width_shift_range=0.1,  # Shift images horizontally by a fraction of the width
    height_shift_range=0.1,  # Shift images vertically by a fraction of the height
    shear_range=0.2,  # Apply shear transformation with a shear angle up to 20 degrees
    zoom_range=0.2,  # Randomly zoom images by a factor up to 20%
    horizontal_flip=True,  # Flip images horizontally
    fill_mode='nearest'  # Fill in newly created pixels after rotation or shifting
)
```

```
X_train, y_train = get_train_data()
X_test, y_test = get_test_data()

X_train = X_train.reshape([-1, 64, 64, 3])
X_test = X_test.reshape([-1, 64, 64, 3])

y_train = label_to_numpy(y_train)
y_test = label_to_numpy(y_test)

history = model_3.fit(
    data_augmentation.flow(X_train, y_train, batch_size=32),
    epochs = 40,
    validation_data = (X_test, y_test),
    shuffle = True,
    callbacks = [monitor])
```

Applying Neural Networks - Results

The model reached a peak validation accuracy of 58.7%

```
Epoch 38/40
211/211 [==============================] - 11s 51ms/step - loss: 0.7753 - accuracy: 0.7048 - val_loss: 1.2123 - val_accuracy: 0.5870
```



Exploring Convolutional Neural Networks

After completing the test run of the model, we have to ask ourselves;

- Is this a good model?
- Is it showing results of overfitting?

Through understanding the issues of the model and how we can improve it, we can use a convolutional neural network in order to improve our analysis of the model.

Shown here, the neural network architecture consists of one convolutional layer for extraction followed by two dense layers for high-level processing and prediction.

```
cnn = Sequential()
cnn.add(Conv2D(64, (3, 3), input_shape=(__, __, __)))
cnn.add(Activation('relu'))
cnn.add(MaxPooling2D(pool_size=(2, 2)))
cnn.add(Flatten())
cnn.add(Dense(units = 128, activation = 'relu'))
cnn.add(Dense(units = NUM_OUTPUTS, activation = 'softmax'))
```

The model reached a peak accuracy of 64.67%. This shows much more improvement over what model was shown before.

```
Epoch 49/50
211/211 [==============================] - 12s 55ms/step - loss: 0.5114 - accuracy: 0.8117 - val_loss: 1.0418 - val_accuracy: 0.6674
Epoch 50/50
211/211 [==============================] - 12s 55ms/step - loss: 0.5001 - accuracy: 0.8154 - val_loss: 1.0360 - val_accuracy: 0.6467
<keras.callbacks.History at 0x79636b977370>
```

Expert Models

So far, we've used models that were made from scratch. However, just training on our data set alone, which is comparatively small to the amount of data in real world, is not enough for our accuracy to be satisfactional.

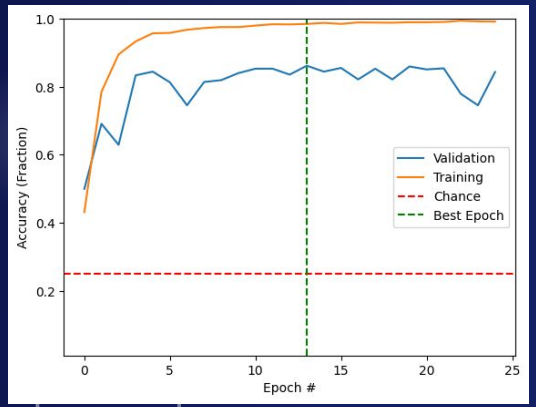
Fortunately, there are a few expert models that have trained on millions of images. We are able to use them for our purposes to get better predictions.

```
transfer_vgg16 = TransferClassifier(name = 'VGG16')
transfer_vgg19 = TransferClassifier(name = 'VGG19')
transfer_resnet50 = TransferClassifier(name = 'ResNet50')
transfer_densenet121 = TransferClassifier(name = 'DenseNet121')
```

We have trained 4 expert models on our data and tested them head-to-head, using the same settings as before (using data augmentation and running 25 iterations).

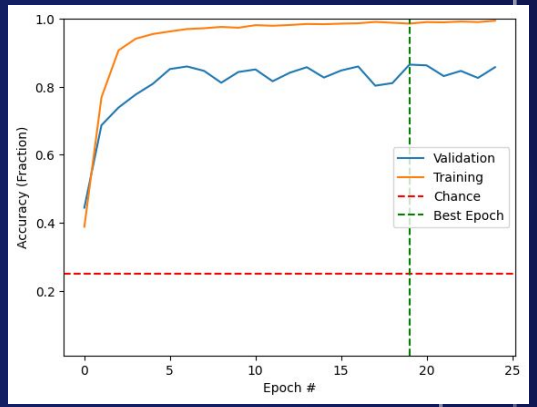
(Trivia 1: these expert models were used on the ImageNet classification problem, where people were challenged to build a model that could distinguish 14 million images contained in more than 20 thousand categories.)
(Trivia 2: the idea of using a model that trained on a different task as a starting point for our model is called **transfer learning** .)

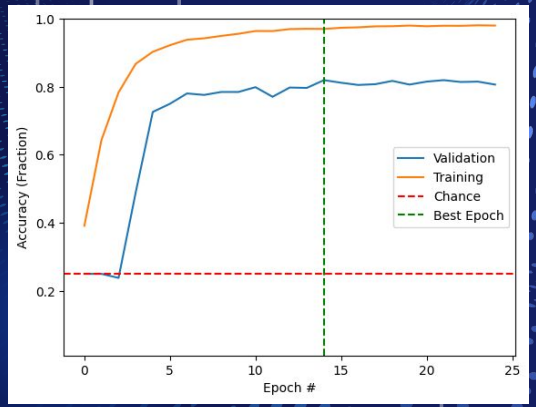
Accuracies



VGG16
(Best: 86.2%)

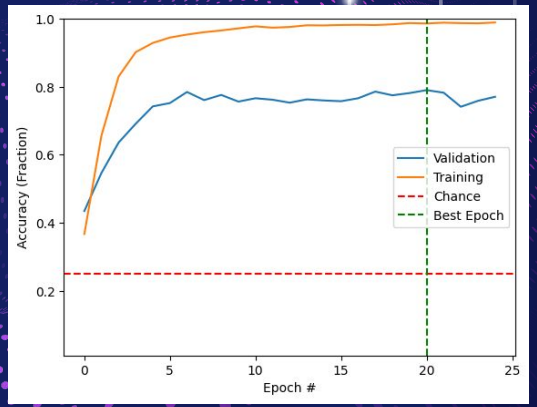
VGG19
(Best: 86.5%)





ResNet50
(Best: 81.9%)

DenseNet121
(Best: 79%)



03

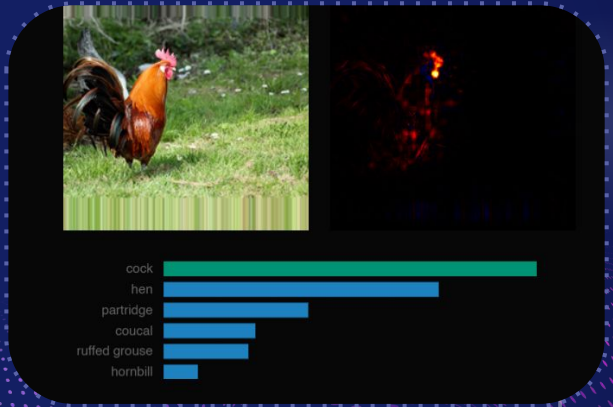
A Closer Look at our Models

What are Salient Maps?

Salient (adj.): influential

We can see which pixels our model more prominently use to classify images.

- The lighter the color the more salient the pixels are in determining what the image is



Using Salient Maps

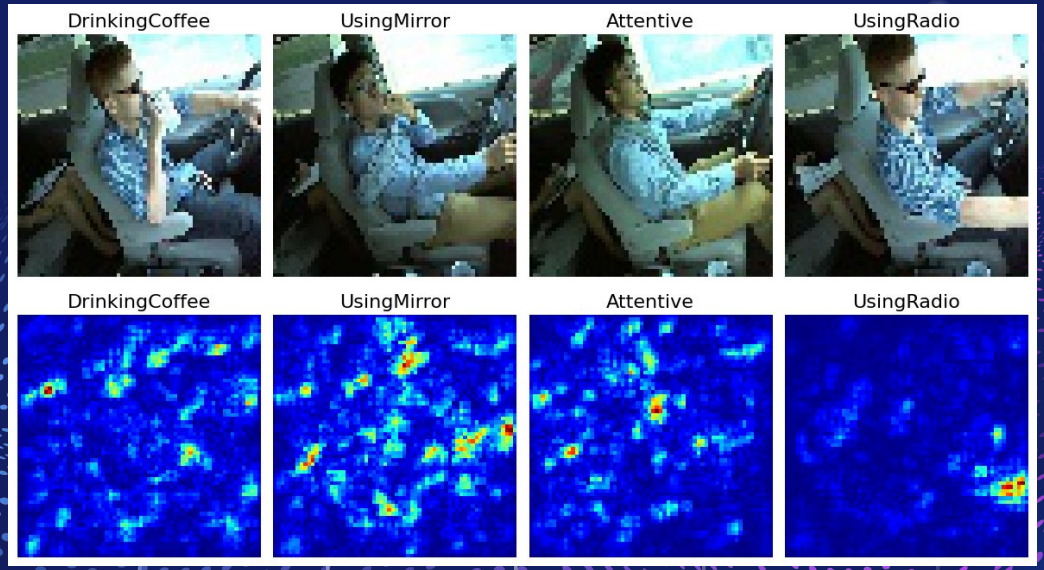
```
X_train, y_train_str = get_train_data(flatten=True)
X_test, y_test_str = get_test_data(flatten=True)

X_train = X_train.reshape([-1, 64, 64, 3])
X_test = X_test.reshape([-1, 64, 64, 3])

# save string versions of labels
y_train = y_train_str
y_test = y_test_str

# convert labels into numpy vectors
y_train = label_to_numpy(y_train)
y_test = label_to_numpy(y_test)

images = []
for i, title in enumerate(y_test_str):
  dim = (64, 64)
  img = np.array(cv2.resize(X_test[i], dim))
  images.append(img)
images = np.asarray(images)

def getImageSamples():
  image_samples = []
  image_samples_labels = []
  idx = random.randint(0, 230)
  for i in range(4):
    image_samples.append(images[idx])
    image_samples_labels.append(y_test_str[idx])
    idx = idx + 230
  image_samples = np.asarray(image_samples)
  return image_samples, image_samples_labels
```

```
def plot_vanilla_saliency_of_a_model(model, X_input, image_titles):
    score = CategoricalScore(list(range(X_input.shape[0])))

    # Create Saliency visualization object
    saliency = Saliency(model, clone=True)

    # Generate saliency map
    saliency_map = saliency(score, X_input)

    # Rendering
    f, ax = plt.subplots(nrows=1, ncols=4, figsize=(12, 4))
    for i, title in enumerate(image_titles):
        ax[i].set_title(title, fontsize=16)
        ax[i].imshow(X_input[i])
        ax[i].axis('off')
    plt.tight_layout()
    plt.show()

    # Plot saliencies
    f, ax = plt.subplots(nrows=1, ncols=4, figsize=(12, 4))
    for i, title in enumerate(image_titles):
        ax[i].set_title(title, fontsize=16)
        ax[i].imshow(saliency_map[i], cmap='jet')
        ax[i].axis('off')
    plt.tight_layout()
    plt.show()
```

Using Salient Maps - Results

Using our previous saliency code, we can visualize our salient pixels.

- We can see that in the UsingRadio category, the hand reaching for the radio is an important determining factor for our model.



Confusion Matrix

"how accurate were we on the distracted images?"

"how much of the attentive drivers were confused for distracted driving?"

"how accurate were we on the attentive category?"

Actual Values

Predicted Values	Positive(1)	Negative(0)
Positive(1)	True Positive (TP)	False Positive (FP)
Negative(0)	False Negative (FN)	True Negative (TN)

When using VGG16, we want to minimize the False Negatives:

- 0 = Attentive Driving
- 1 = Distracted Driving

Confusion Matrix

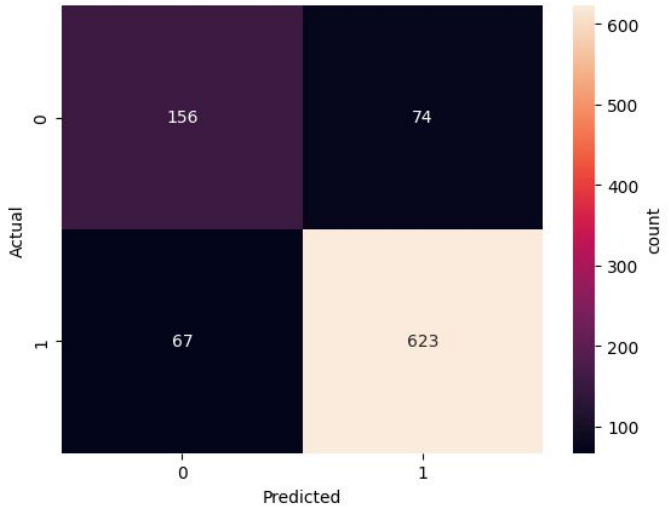
First, we separate the Attentive labels with the others(UsingRadio, UsingMirror, DrinkingCoffee) into a binary classification with 0 and 1.

```
final_labels = []
for label in test_labels_strings:
  ### FILL IN THIS LOOP ###
  if label == 'Attentive':
    final_labels.append(0)
  else:
    final_labels.append(1)
binary_predictions = []
for label in predictions:
  if label == 0:
    binary_predictions.append(0)
  else:
    binary_predictions.append(1)

print('Accuracy is %d %%'%(accuracy_score(final_labels, binary_predictions)*100.0))
```

```
Accuracy is 84 %
```

Then, we can see the overall accuracy and the amount of TPs, FPs, FNs, and TNs. We can see from our chart that we have the least amount of FNs, which is good.

```
# grab our plotting package
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(confusion, annot = True, fmt = 'd', cbar_kws={'label':'count'});
plt.ylabel('Actual');
plt.xlabel('Predicted');
```



Precision and Recall

Percentage of positive identifications that were actually positive

Precision = TP/ (TP+TN)

0.893830703012912 5

Percentage of positives that were identified correctly

Recall = TP/ (TP+FN)

0.9028985507246376

Conclusion: There are more TN than FN